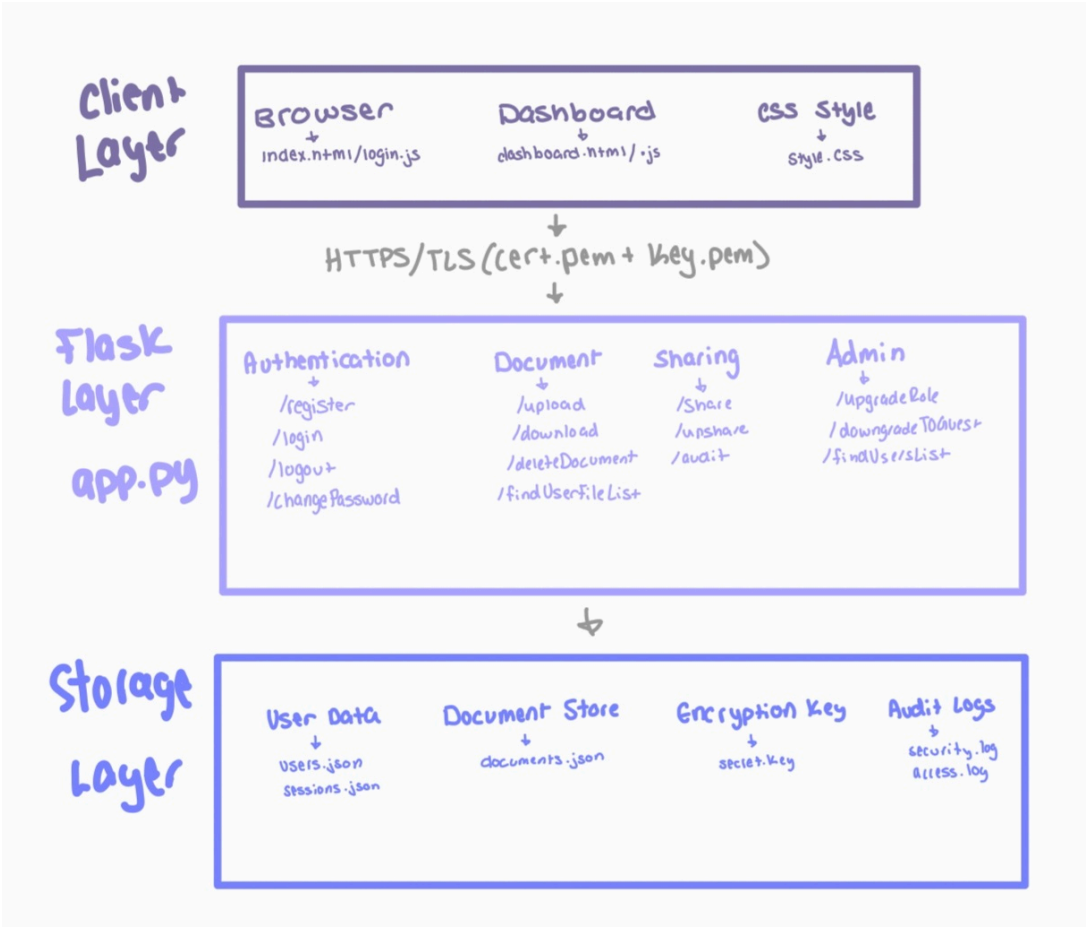
Security Architecture

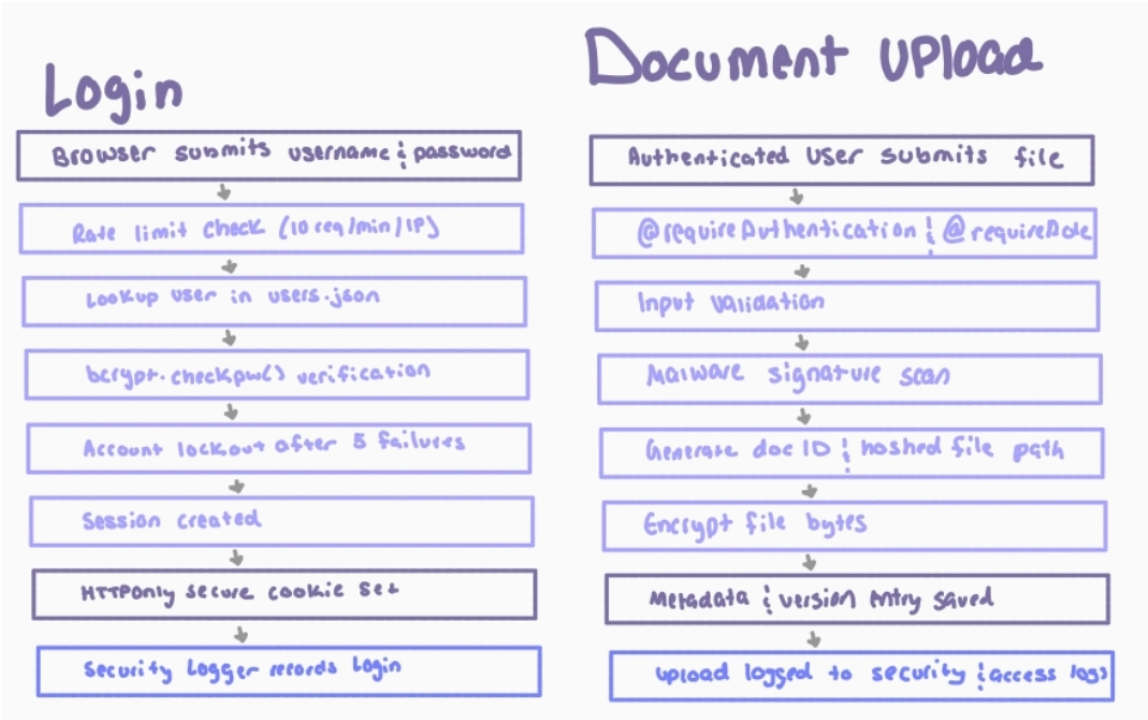
Maria Vaikath(mav218) & Rithika Pathuri (rrp149)

A. Architecture Overview (10 points)

1. System architecture diagram



2. Data flow chart



Login

Step	Action	Security Control
1	Browser submits credentials	HTTPS transport encryption; username length-checked (max 20)
2	Rate limit check	Max 10 requests/min/IP tracked in trackerForIPs dict; 429 on breach
3	User lookup	users.json read; non-existent user returns generic error (no username enumeration)
4	Lockout check	locked_until compared to time.time(); 401 returned if still locked
5	Password verification	bcrypt.checkpw() with cost factor 12; timing-safe comparison
6	Failure handling	failed_attempts incremented; locked for 900s after 5 failures; logged
7	Session creation	secrets.token_urlsafe(32); stored with IP + User-Agent
8	Cookie issuance	HttpOnly, Secure, SameSite=Strict, max_age=1800

9	Audit	LOGIN_SUCCESS/FAIL logged with IP, timestamp, username

Upload Document

Step	Action	Security Control
1	Authenticated user submits file	Session cookie validated by loadUserSession() before handler runs
2	Role authorization	@requireRole(ROLE_ADMIN, ROLE_USER) blocks guests
3	Filename sanitization	secure_filename(); filename length capped at 100 characters
4	Extension whitelist	Only .pdf, .doc, .docx, .txt accepted — all others denied
5	MIME type check	file.mimetype validated against allowed set — rejects mismatches
6	Content scan	Raw bytes checked for and signatures
7	Size enforcement	Rejected if file exceeds 10 MB (MAX_FILE_SIZE)
8	Secure storage path	SHA-256(uuid + PEPPER) — unguessable encrypted filename
9	Encryption	Fernet.encrypt(file_bytes) written to disk; key stored in secret.key
10	Metadata + audit	documents.json updated; UPLOAD logged to both log files

3. Component descriptions

app.py → Main application

- Contains all Flask route handlers, middleware hooks (before_request / after_request), and decorator-based access control logic.
- Implements:
  - authentication routes (/register, /login, /logout, /changePassword)

        - document management routes (/upload, /download, /share, /unshare, /deleteDocument)
        - administrative routes (/upgradeRole, /downgradeToGuest, /findUsersList)
        - audit/listing routes (/findUserFileList, /document/<id>/audit)

    classes.py → infrastructure

    - Provides 5 classes
        - EncryptedStorage - Fernet symmetric encryption/decryption for file blobs
        - SessionManager — Token creation, validation, expiry, and destruction
        - SecurityLogger — Structured JSON event logging to security.log
        - AccessLogger — Access event logging to access.log
        - DocumentManager — Metadata CRUD, versioning, sharing, and audit trail management

    login.js/dashboard.js → frontend

    - login.js handles registration and login form submissions via the Fetch API over HTTPS, with inline toast notifications for all error states.
    - dashboard.js manages
        - file list view
        - user management (admin only)
        - document sharing modal
        - audit trail modal
        - password change flow

    via authenticated Fetch API calls using the session_token cookie.

4. Technology stack justification

Technology	Choice	Justification
Backend	Python / Flask	Lightweight and explicit. Decorator pattern maps cleanly to per-route security controls. Rich security library ecosystem (bcrypt, cryptography).

Password hashing	bcrypt (rounds=12)	Purpose-built for passwords; adaptive cost factor; built-in salt. Preferred over MD5/SHA for resistance to GPU-based attacks.
Symmetric encryption	cryptography.Fernet	AES-128-CBC + HMAC-SHA256 in one API. Authenticated encryption prevents tampering. Well-audited, NIST-aligned.
Session tokens	secrets.token_urlsafe(32)	256 bits of CSPRNG entropy. Python's secrets module is explicitly designed for security tokens, unlike random.
Transport security	TLS via OpenSSL self-signed cert	Encrypts all data in transit. RSA-4096 chosen for strong key length. Self-signed acceptable for academic; CA-signed in production.
Data storage	JSON flat-files	Eliminates SQL injection attack surface. Meets spec (no DB required). Simplifies deployment with no external dependencies.
Frontend	Vanilla JS	No framework dependencies reduces third-party vulnerability surface. Full control over how data is inserted into DOM (preventing XSS).

B. Threat Model (10 points)

1. Asset identification
    - User Credentials
        - Usernames, hashed passwords, email addresses stored in users.json
    - Session Tokens
        - 32-byte URL-safe tokens in sessions.json — authenticate all requests
    - Document Contents
        - Fernet-encrypted files on disk in data/files/

- Document Metadata
  - Owner, sharing rules, versions, audit log in documents.json
- Encryption Key
  - secret.key — compromise exposes all stored document content
- Audit Logs
  - security.log + access.log — forensic record of all security events
- User Privacy
  - Email addresses and usernames used to identify individuals
- Application Source
  - app.py, classes.py - security logic and pepper values

2. Threat enumeration

Threat	Target	Attack Description
Credential Brute Force	User Credentials	Repeated POST /login with password variations to guess valid credentials
Session Hijacking	Session Tokens	Steal session_token cookie via XSS or network sniffing and replay it
Username Enumeration	User Credentials	Probe /login with known usernames to confirm existence via response differences
Path Traversal	Filesystem	Inject ../../etc/passwd in filename parameter to read arbitrary files
Malicious Upload	Document Store	Upload disguised script (e.g., .php as .pdf) to achieve server-side execution
CSRF	Session / Actions	Trick authenticated user into making unintended requests from attacker-controlled page
Privilege Escalation	User Roles	Tamper with API request body to assign admin role to own account
Log Tampering	Audit Logs	Modify or delete log entries to erase evidence of unauthorized actions

IDOR on Documents	Document Contents	Guess/enumerate docIDs to access other users' documents without authorization
Encryption Key Theft	Encryption Key	Read secret.key from filesystem to decrypt all stored documents
XSS Data Exfiltration	Session Tokens	Inject script that reads cookies or page data and sends to attacker server
Login Flood	Application Availability	Flood /login to exhaust rate limiter, lock accounts, or degrade server performance
Upload Bomb	Storage / Memory	Upload max-size files repeatedly to exhaust disk or memory
Forced Browsing	Admin Routes	Access /findUsersList or /upgradeRole directly without admin session

3. Vulnerability assessment

Vulnerability	
Brute-force login	rate limiter resets on restart
IDOR on document download	Mitigated by @requireDocumentPermission + UUID docIDs
Session token not IP-pinned	Validated IP check commented out in SessionManager
secret.key on local filesystem	Not in version control; high impact if server compromised
CSP allows 'unsafe-inline'	Necessary for inline scripts; refactor needed for production
Rate limiter in-memory (no persistence)	Restart clears tracker; persistent store would be stronger
Malicious upload (limited sig check)	Only checks <script> and <?php — advanced polyglots possible
No concurrent session limit	Multiple simultaneous sessions allowed per user

User role stored in localStorage	UI-only risk; all auth decisions enforced server-side

4. Attack scenarios
    - Brute Force
        - Gain access to target user's account by guessing the password
            1. Attacker sends POST /login with correct username and varying passwords
            2. After 10 requests in 60 seconds from same IP, rate limiter returns 401
            3. Attacker rotates IP (e.g., via proxies)
            4. After 5 failed attempts on the account, locked_until is set for 900 seconds.
            5. Attacker must wait or target a different account
        - Defense: IP rate limiting (10/min) + per-account lockout (5 attempts → 15 min)
    - IDOR Document Access
        - Download another user's private document without authorization
            1. Attacker logs in as a valid low-privilege user.
            2. Attacker guesses or enumerates a docID
            3. Attacker calls GET /download/<guessed_docID>.
            4. @requireDocumentPermission checks: attacker is not owner, not in sharedWith.
            5. 403 returned; UNAUTHORIZED_ACCESS logged as CRITICAL.
        - Defense: UUID DocsIDs; @requireDocumentPermission on every download
    - Malicious File Upload
        - Upload a web shell or malware disguised as a document
            1. 1. Attacker renames malware.php to malware.pdf.
            2. POST /upload with Content-Type: application/pdf.
            3. Server checks extension (.pdf — allowed).
            4. Server checks MIME type (file.mimetype — allowed if declared by client).
            5. Server checks raw bytes for <script> and <?php — PHP shell blocked.
            6. File encrypted with Fernet and stored — not served as executable
        - Defense: Extension whitelist + MIME check + signature scan + Fernet encryption means the file is never executed
    - Privilege Escalation via Forced Browsing
            1. Attacker is logged in as ROLE_USER.

        2. Attacker directly calls POST /upgradeRole with JSON body {username: 'attacker', role: 'admin'}.
        3. @requireRole(ROLE_ADMIN) runs — user's stored role is 'user'.
        4. ACCESS_DENIED logged at WARNING
        5. 403 returned.
    - Defense: @requireRole decorator reads role from users.json (server-side), not from the request or localStorage
5. Risk prioritization

Rank	Risk	Rationale
1	Brute-force login (distributed)	Rate limiter is in-memory only; persistent Redis store needed in production
2	CSP unsafe-inline	Weakens XSS protection; nonce-based CSP would be strongest
3	Session not IP-pinned	Stolen token usable from any IP; IP binding would limit this window
4	In-memory rate limiter	Clears on restart; attacker can trigger restart to reset
5	secret.key local storage	Key management should use environment variables or secrets vault
6	Limited upload signature scan	Polyglot files could evade current check; python-magic would improve
7	No concurrent session limit	Credential sharing undetectable; session count limit would help
8	Role in localStorage	Low risk — informational UI only; server never trusts it

C. Security Controls (10 points)

1. User Authentication

Description: Ensures only legitimate, registered users can access the system. Covers registration validation, password hashing, login verification, account lockout, and rate limiting.

Implementation: Registration → Password hashing → Login verification → Rate limiting → Account lockout

Testing Methodology:

- Register with an invalid username
- Register with an invalid password
- Submit 11 login attempts within 60 seconds from the same IP
- Submit 5 wrong passwords for one account
- Attempt logging while the account is locked
- Verify users.json contains only bcrypt hashes

Known Limitations:

- IP rate limiter lives in memory
- Distributed attacks across many IPs bypass the per-IP limit
- No CAPTCHA or 2FA

Mitigation Strategies:

- Replace in-memory dict with Redis to persist rate limit counters across restarts
- Add TOTP as 2FA
- Implement exponential backoff in addition to hard lockouts

2. Access Control (RBAC)

Description: Role-Based Access Control enforces that users can only perform actions permitted by their role (Admin, User, Guest) and their relationship to a specific document.

Implementation: System roles → @requireRole decorator → @requireDocumentPermission decorator → Admin bypass → Role management

Testing Methodology:

- Log in as Guest and attempt a POST /upload
- Log in as User and attempt GET /findUsersList (Admin only)
- Log in as User A and attempt GET /download on User B's document
- Log in as User and attempt to call /upgradeRole
- Log in as Admin and access User B's document (verify bypass)
- Attempt to share a document while holding only a Viewer role

Known Limitations:

- Role is stored in client localStorage (UI visibility only)
- No role expiry for temporary elevated privileges

Mitigation Strategies:
- Add role expiry timestamps for temporary grants
- Log all role changes and require admin-to-admin confirmation

3. Input Validation & Injection Prevention

Description: Validates all user-supplied input server-side and subjects file uploads to a multi-stage validation pipeline while using auto-escaping to prevent XSS.

Implementation: Field validation (Regex/Length) → File upload pipeline → XSS prevention (Jinja2/CSP) → Path traversal protection → CSRF protection (SameSite=Strict)

Testing Methodology:
- Submit a username containing a <script> tag
- Upload a file with a path traversal string (../../) in the filename
- Upload a file with a forbidden extension (.exe)
- Upload a file with mismatched MIME types and extensions
- Upload a file containing PHP or Script signatures
- Send an oversized filename or forged session cookie

Known Limitations:
- Signature scan only checks for basic script/PHP patterns
- MIME type is taken from request rather than file magic bytes
- CSP allows 'unsafe-inline' for scripts

Mitigation Strategies:
- Use python-magic to independently verify MIME types
- Expand signature scan to cover polyglot files and PE headers
- Refactor inline scripts and switch to nonce-based CSP

4. Encryption

Description: Protects data in transit via TLS and data at rest using Fernet symmetric encryption for all uploaded document content.

Implementation: Transport (TLS/HSTS) → Fernet encryption (AES-128-CBC) → Storage path obfuscation → Secure session tokens

Testing Methodology:

- Access the site via HTTP and verify redirect to HTTPS
- Inspect headers for Strict-Transport-Security
- Read a file directly from the storage directory to verify ciphertext
- Verify secret.key is present, 44-bytes, and git-ignored
- Modify a byte of an encrypted file and attempt a download

Known Limitations:

- Self-signed certificate triggers browser security warnings
- Encryption key is stored as a local file on the server
- No mechanism for key rotation
- Mitigation Strategies:
- Replace self-signed cert with Let's Encrypt CA-signed certificate
- Store encryption keys in an environment variable or Secrets Manager
- Implement annual key rotation and re-encryption

5. Session Management

Description: Issues secure 256-bit session tokens upon login that expire after 30 minutes of inactivity and are destroyed upon logout.

Implementation: Token generation (CSPRNG) → Session validation hook → Cookie attributes (HttpOnly/Secure/SameSite) → Session destruction

Testing Methodology:

- Wait 31 minutes and attempt an authenticated request
- Log out and attempt to replay the captured session token
- Attempt to access session_token via document.cookie in console
- Verify independent sessions by logging out of one browser only
- Verify token length and format in sessions.json

Known Limitations:

- IP binding is currently disabled (allows token replay)
- No limit on concurrent sessions per user
- Session data is not encrypted at rest in sessions.json

Mitigation Strategies:

- Re-enable IP binding for non-mobile clients
- Add a maximum concurrent session limit with oldest-first eviction
- Encrypt sessions.json using the existing EncryptedStorage class

6. Security Headers

Description: Applies seven distinct HTTP security headers to every response to protect against clickjacking, MIME-sniffing, and XSS.

Implementation: Content-Security-Policy → X-Frame-Options → X-Content-Type-Options → X-XSS-Protection → Referrer-Policy → Permissions-Policy → Strict-Transport-Security

Testing Methodology:

- Inspect response headers via curl -I
- Verify A/A+ rating on securityheaders.com
- Attempt to embed the application in an external iframe
- Try loading an external script to verify CSP blocking

Known Limitations:

- CSP 'unsafe-inline' weakens script protection
- HSTS on a self-signed cert can cause browser lockout

Mitigation Strategies:

- Move inline JS to external files and use a nonce-based CSP
- Set HSTS preload once a valid CA certificate is deployed

7. Logging & Monitoring

Description: Captures security-significant events and user actions in structured JSON logs for audit and incident response.

Implementation: SecurityLogger (Auth/Access) → AccessLogger (Operations) → Structured Log schema → Startup configuration logging

Testing Methodology:

- Trigger a failed login and verify the JSON entry in security.log
- Verify ACCOUNT_LOCKED events are logged with ERROR severity
- Verify ACCESS_DENIED events are logged with CRITICAL severity
- Check that log files are append-only and not overwritten

Known Limitations:

- Logs are plain text files without tamper detection
- Lack of log rotation causes unbounded file growth
- No real-time alerting for CRITICAL events

Mitigation Strategies:

- Forward logs to an append-only store or SIEM (ELK/Splunk)
- Implement log rotation using RotatingFileHandler
- Add webhook or email alerts for high-severity events

D. Data Protection(10 points)

1. Data Classification

Classification	Data Elements	Protection Requirement
Critical	Fernet encryption key (secret.key), raw document contents before encryption	Must never be stored unprotected; key must not appear in logs, source code, or version control
Confidential	bcrypt password hashes, session tokens, uploaded file ciphertext, email addresses	Stored only in protected files; never transmitted in plaintext; never logged in full
Sensitive	Document metadata (ownership, sharing, versions), audit log entries, IP addresses in logs	Access restricted to authenticated users with appropriate role; logs append-only
Internal	Usernames (display names), document filenames (after authorization), role names	Available to authenticated users; filenames visible only after permission check

2. Encryption Methods

- Transport Encryption
  - TLS 1.2+ via Flask ssl_context
  - require_https() before_request hook redirects HTTP → HTTPS (301). HSTS header prevents downgrade attacks
- Data-at-Rest Encryption
  - AES-128-CBC with HMAC-SHA256 (Fernet specification)
  - Authenticated encryption — HMAC tag verifies integrity. Any modification to ciphertext raises InvalidToken on decryption.
- Password Hashing
  - bcrypt with cost factor 12
  - Unique per user via bcrypt.gensalt(rounds=12) — stored embedded in the hash string

3. Key Management

- Fernet Symmetric Key
  - Fernet.generate_key() on first run if secret.key absent

    - Read at application startup by Flask process only; OS file permissions should be restricted to the app user
    - Read once at EncryptedStorage.__init__() on application startup
- Session Token
    - secrets.token_urlsafe(32) — 256 bits from OS CSPRNG on each successful login
    - Logout calls destroySession() immediately. Login destroys old token before creating new one.
    - Rotated on login/logout

4. Secure Deletion Procedures

- Document Deletion: when a document is deleted vis POST /deleteDocument
    1. Owner or admin authorization verified (isOwner() check + @requireRole fallback)
    2. All version file paths retrieved from documents.json metadata
    3. os.remove(filePath) called for each version — removes the encrypted ciphertext blob from the filesystem
    4. Document entry deleted from the in-memory metadata dict (del metadata[docId])
    5. Updated metadata written back to documents.json via saveMetadata()
    6. DELETE action appended to the document audit log before metadata removal
    7. DELETE_SUCCESS logged to security.log and access.log
- Session Deletion: on logout or session expiry:
    8. destroySession(token) deletes the token key from sessions.json
    9. sessions.json re-serialized to disk with token absent
    10. Response cookie cleared with empty value and expires=0